

Experiment 4.3

Title

Concurrent Ticket Booking System with Seat Locking and Confirmation

Objective

Learn how to implement a ticket booking system in Node.js that handles concurrent seat reservation requests safely using a seat locking mechanism. This task helps you understand how to manage in-memory state, handle concurrent access, and design a system that prevents race conditions during booking.

Task Description

Create a Node.js and Express.js application that simulates a ticket booking system for events or movie theaters. Implement endpoints to view available seats, temporarily lock a seat for a user, and confirm the booking. Design a seat locking mechanism so that when a seat is locked, it cannot be locked or booked by other users until it is either confirmed or the lock expires automatically (for example, after 1 minute).

Store seat states in an in-memory data structure for simplicity. Include clear success and error messages for different scenarios, such as trying to lock an already locked or booked seat, or confirming a seat without a lock. Test your API by simulating concurrent requests to demonstrate that the locking logic correctly prevents double booking and ensures reliable seat allocation.

Expected Output

The screenshot shows a REST client interface with a GET request to `http://localhost:3000/seats`. The response is a 200 OK status with a 9 ms response time and 499 B of data. The response body is a JSON array of 7 seat objects, each with an index and a state.

```
1 {
2   "1": {
3     "state": "available"
4   },
5   "2": {
6     "state": "available"
7   },
8   "3": {
9     "state": "available"
10  },
11  "4": {
12    "state": "available"
13  },
14  "5": {
15    "state": "available"
16  },
17  "6": {
18    "state": "available"
19  },
20  "7": {
```

POSTGet data

My Collection / Get data

POST

http://localhost:3000/LOCK/5

Params

Authorization

Headers (7)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

Content-Type: application/json

Body

Cookies

Headers (7)

Test Results (1/1)

200 OK

{}

JSON

Preview

Visualize

1

{

2

"message": "Seat 5 locked successfully. Confirm within 1 minute."

3

}

POSTGet data

My Collection / Get data

POST

http://localhost:3000/CONFIRM/5

Params

Authorization

Headers (7)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

Content-Type: application/json

Body

Cookies

Headers (7)

Test Results (1/1)

200 OK • 6 ms • 276 B •

{}

JSON

Preview

Visualize

1

{

2

"message": "Seat 5 booked successfully!"

3

}

POST Get data

My Collection / Get data

POST http://localhost:3000/CONFIRM/2

Params Authorization Headers (7) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

Body Cookies Headers (7) Test Results (0/1)

400 Bad Request

{ } JSON Preview Debug with AI

```
1 {
2   "message": "Seat is not locked and cannot be booked"
3 }
```